

On this page

Authentication

Feedzai allows you to authenticate and protect your data through JSON Web Tokens (JWTs). A JSON Web Token (JWT) is a piece of information that is transmitted as a JSON object from clients to Feedzai, and its purpose is to ensure that client requests (e.g. transactions for processing) have not been compromised.

These tokens are generated using the client's private key, which can be verified with the corresponding public key. The public key must be shared with Feedzai beforehand to verify that the token sent with the request belongs to the client.

Overview

JWTs protect against different attacks using the following mechanisms:

- **Encryption/signing** so that only authenticated users can access sensitive data.
- **Expiration dates** to protect against replay attacks.
- **JWT revocation** to block exposed JWTs as soon as possible.

You can see more details on integrating JWTs with your software in the sub-sections below, but in general you will:

1. Generate a private-public key pair to sign the JWT.
2. Provide the public key to Feedzai in [JSON Web Key Set](#) format.
3. Frequently generate new JWTs with a reduced validity period (e.g. 5 minutes). Each JWT is signed with your private key generated in step 1.
4. For each request to Feedzai, include a JWT in the `Authorization` header.

Given the recommended algorithm and parameters, you can choose the tool that best fits your system. A comprehensive list of tools is available at the [libraries section of jwt.io](#).



Third-party security vulnerabilities: When choosing a library, make sure it has no known security vulnerabilities.

JSON Web Key

A key pair is necessary to cryptographically sign and verify the JWTs sent with your requests.

- You will use the private key to sign your JWTs.
- Feedzai will use the public key to verify the signature, and therefore the authenticity of the requests.

To create a JWK set, you need to choose the signing algorithm and the JWK parameters. Feedzai recommends the `RS512` algorithm (`RSASSA-PKCS1-v1_5` using SHA-512) with a 4096-bit key. See the sections below to learn how to generate a JWK set with standard tools.



Beware of private key parameters: Construct the JWK set so that you **do not** send any of the private key parameters.

Your JWK set should look similar to this:

```
{
  "keys": [
    {
      "e": "AQAB",
      "n": ...
    }
  ]
}
```

```
"187EwmXQwXi08P8bqS0Se0iy0yJGsojjmeL6qwLLjTBfxxeNgTtivzG8YdCgrLnBQWy4j9FqQ5wbqy0wf6CBv6xj9ZnqyyS
cle7ubAKRt4RKtv4yrxVb1g5ZwcfD6yWDIH1jRy5oGQEWrg9918Z730545lYe8rMDiWkFPybt1vfGBdGqbvm374XNfLRb0s
F7cPCRCBa3Vdzmg5BgoN9kFUXIvFKjXdgZQGEIEonLL8ZSmqg993VBVL0Ph020iJ91DVvb9JMIFAqR0HQNyzchb0WgEPUKoR
BRDXSCus9ZSpHuU7iL7qKLaQdPy01ffIG0o7kf6allFG60BcnzqX8r",
  "kty": "RSA",
  "kid": "a28db6df-f2f4-4267-be46-371502768aa1sig",
  "use": "sig"
}
}
}
{
  "kid": "mykey2",
  "kty": "RSA",
  "alg": "RS512",
  "use": "sig",
  "e": "AQAB",
  "n": "k51ZPyu9gocn508axzob_hSKaIANjApY4pY3h1oym1Z15-6DN3WxB3YHUyXPRd-iBzF7177XjQAwTzAaz-
PKLZ4bC52noanLx7Ihyf3nQA0whEv16zaJLI-
HqMEMp6JKPjKJ8LkanVnvxWNI0zUBwcB0wt9cxph0evyV094cXA0RZxUZxa7C3FuSYk02zWorMH0ZkC3harkJx0C9Q7nBiGw
PQmB8ZkLx04iWlm8ldyxDJPKL-PC4-
gS3y2mgPfZVAaK_VGod88PUMf4vFax7a03v2VLtn9wm8UWs9SXATxJ5cExgIIVfKdsZAcS3hWnhUe30f4zAEbKh2Ah5DtFY9
xvtsU8Fglak7wLcWVnubWwglI2hmFvFU7Jsyzb_L-
jD7XCyrg7QuFV8mX2jVcTbsSxnd8GLRNbvmztvsjfaQHxEHraTRJFoG01Y3JXD1Y8Kj400QXxBNXaFxDa9lBfzWfDkgWvcy
eiTPeWpKc6cS1zcDaQNRzZr3aZR-
gI4U8YTAXcIKDdom2GCQSx9WfmFq3uAj95duL5CgyGfw7L-02c6ckRr6kBAKebURkSsZX0yT08bdbS8GzPMD59CrRL2SnJJJe
qHyrEav8JCu0x3UCXbyqlfqQgzVsXnpFUMIS-sUhe3nyVQ0eJ_WoYq7oAbjLpx_Bw5elQJf4zJbJd2Pk"
}
```

Copy

In the aforementioned example, the following parameters are used:

Name	Extended name	Description
e	Exponent	RSA public exponent e.
n	Modulus	RSA public modulus n.
kty	Key Type	Should be <code>RSA</code> to symbolize you are using the RS512 algorithm.
kid	Key Id	Should specify a key identifier of your choice.
use	Public Key Use	Should be <code>sig</code> to note this key will be used to verify JWTs.

How to generate a key-pair and JWK

- 1. [Download Feedzai's JWT Generator](#).
- 2. Unpack the tar.gz artifact.
- 3. Check generator options:

```
./bin/generate-asymmetric --help
```

Copy



Make sure you repeat steps 1 through 3 for a total of two private-public key pairs: one for `prod` and another one for `qa`.

- 4. Run the JWT Generator using the following command. Adjust the parameter values according to your needs:
- ```
./bin/generate-asymmetric \
--mode JWK \
--generateKeys \
--privateKey "/path/to/privateKey" \
```

```
--publicKey "/path/to/publicKey" \
--kid "key_id"
```

Copy

The JWK Set JSON will be displayed on your standard output. Alternatively, you can use the `--jwk` parameter to specify a file where it should be stored instead.

5. Store the generated public and private key files securely as they will be required to generate JWTs. **Never share your private key.**
6. [Manage JWKs](#) in Pulse.

## JSON Web Tokens

Once you have a private key, you can generate your JWTs. All JWTs are composed of a set of headers, a set of claims and a signature.

### JWT Headers

Besides the algorithm ( `alg` ), the only mandatory header for JWTs is the `kid` (the key ID), which indicates which key was used to sign the token and **must** match a valid key in the JWK Set used for validation. The set of headers should be similar to:

```
{
 "kid": "keyID",
 "alg": "RS512"
}
```

Copy

### JWT Claims

Typically, the following claims should be present on the JWTs:

| Claim name        | Extended name   | Description                                                                                                                 |
|-------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------|
| <code>iss</code>  | Issuer name     | Identifies the issuer of the JWT. This value will be provided by Feedzai, and it should match the tenant id, if applicable. |
| <code>exp</code>  | Expiration Time | Date after which the JWT must be rejected.                                                                                  |
| <code>nbft</code> | Not before Time | Date before which the JWT must be rejected.                                                                                 |
| <code>jti</code>  | JWT id          | Identifier used to revoke JWTs.                                                                                             |
| <code>sub</code>  | Subject         | Identifies the user of the JWT.                                                                                             |

Your final set of claims should be similar to:

```
{
 "iss": "feedzai",
 "exp": 1523022355,
 "nbft": 1523022055,
 "jti": "a00cd6dd-4e55-4694-aeb5-4050e7d063b6",
 "sub": "700e55f9-15b0-4f95-8094-960138a0d886"
}
```

Copy

## How to generate a JWT

1. [Download Feedzai's JWT Generator](#).
2. Unpack the tar.gz artifact.
3. Check generator options:

```
./bin/generate-asymmetric --help
```

Copy

4. Run the JWT Generator using the following command. Adjust the parameter values according to your needs:

```
./bin/generate-asymmetric \
--mode JWT \
--privateKey "/path/to/privateKey" \
--publicKey "/path/to/publicKey" \
--iss "issuer" \
--sub "subject" \
--exp "$((EPOCHSECONDS + 300))" \
--kid "key_id"
```

Copy

The JWT will be displayed on your standard output. Alternatively, you can use the `--jwt` parameter to specify a file where it should be stored instead.

## How to send the signed JWT🔒🔒



Note that you can only send the signed JWT after the corresponding JWK is uploaded to Pulse (see [Manage JWKs in Pulse](#)).

Send the signed JWT in the `Authorization` header of HTTP requests made to Feedzai's API, as follows:

```
Authorization: Bearer <your_jwt>
```

Copy

## Manage JWKs in Pulse🔒🔒

Once you create the JWK set, the following JWK steps must be taken:

1. Log into Pulse and confirm the current [list](#) of Authorization keys. Before revoking any key, make sure you are no longer using it.
2. [Add](#) the new key to Pulse.
3. Start signing messages with JWTs based on the new key.
4. [Revoke](#) the old key.
5. [Delete](#) revoked Authorization keys.

## List Authorization keys🔒🔒

To list the existing Authorization keys, access the **Administration** page and select the **Authorization Keys** tab on the sidebar.

Select a label name or expand to see the Authorization keys associated with that label:

## Add Authorization keys🔒🔒

To add a new Authorization key, select a label name and upload the JSON file that contains the new key:



The new key must be associated with an existing label.



Make sure you start signing the new messages with the new key.

## Revoke Authorization keys<sup>ⓘ</sup>

The following animation illustrates how to add a new key and revoke the current one.



An Authorization key should only be revoked once a new Authorization key has been added (see [previous step](#)) and JWTs are already being signed with the new key.

## Delete revoked Authorization keys<sup>ⓘ</sup>

By deleting an Authorization key, you are permanently removing it from the list of keys. Only revoked keys can be deleted.